

CSCI 4061 • LAB 03

Catching SIGINT Gracefully

`sigaction()`, signal handlers, `SA_RESTART`, and a global flag



UNIVERSITY OF MINNESOTA

Driven to Discover®

Today's plan

- 01 Signals recap — what actually arrives when you press Ctrl-C
- 02 Why catch one — Ctrl-C throws away `wc_signal`'s counts
- 03 `sigaction()` — the struct, its fields, and the call
- 04 `SA_RESTART` and the global flag — telling `main()` to stop, safely
- 05 Putting it together — what `wc_signal.c` should do

Signals, a quick recap

Last week the shell sent signals to our programs. This week the program itself decides what happens when one arrives — for most signals, it gets a say.

Signal	Sent by	Default effect	Can we catch it?
SIGINT	Ctrl-C	Terminate	Yes — today's lab
SIGTSTP	Ctrl-Z	Stop (pause)	Yes
SIGTERM	kill <pid>	Terminate	Yes
SIGKILL	kill -9 <pid>	Terminate	No — never
SIGSTOP	kill -STOP <pid>	Stop (pause)	No — never

Try it: wc_signal, before the fix

```
while (keep_going) {
    int cur_char = fgetc(stdin);
    if (cur_char == EOF) {
        keep_going = 0;
    } else {
        num_chars++; // ...
    }
}
printf("%d lines, %d words, ...");
```

`seq 10 | ./wc_signal`

input ends (EOF) = loop exits, counts are printed

`yes | ./wc_signal`

input never ends = the loop runs forever

Ctrl-C

default SIGINT action kills it = the counts are never printed

PART 2

sigaction()

Installing your own signal handler

struct sigaction: three fields to fill

```
struct sigaction {  
    void      (*sa_handler)(int); // what to run  
    sigset_t   sa_mask;           // what to block meanwhile  
    int        sa_flags;          // options  
};
```

Field	What you put there
sa_handler	Your function, e.g. handler — or SIG_IGN (ignore) / SIG_DFL (default)
sa_mask	Signals blocked while the handler runs; sigfillset() blocks them all
sa_flags	Behavior options, e.g. SA_RESTART

Installing a handler, step by step

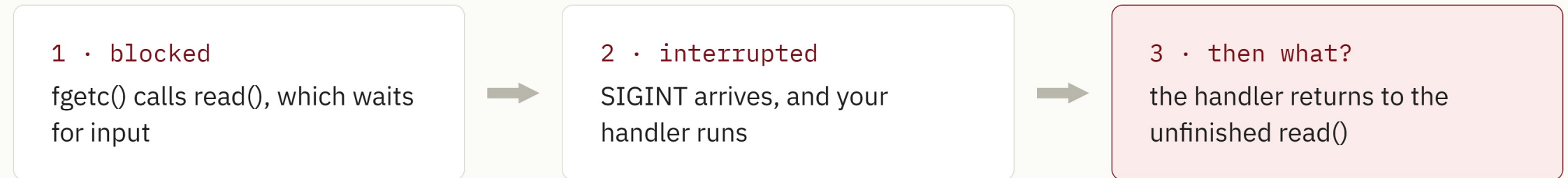
```
void handler(int signum) { ... }

struct sigaction sa = {0};
sa.sa_handler = handler;           // 1
sigfillset(&sa.sa_mask);           // 2
sa.sa_flags = SA_RESTART;          // 3
if (sigaction(SIGTERM, &sa, NULL) == -1) { // 4
    perror("sigaction");
    return 1;
}
```

- 1 Point sa_handler at your function
- 2 Block other signals while it runs
- 3 Restart interrupted system calls
- 4 Register it for one signal; check for -1

This example handles SIGTERM — in the lab, you'll do the same for SIGINT.

Why SA_RESTART?



sa_flags	What read() does after the handler
0	Fails with errno = EINTR — the caller must notice and retry
SA_RESTART	Restarted automatically; your code never sees the interruption

Handler to main(): a global flag

```
volatile sig_atomic_t running = 1;
// global: handler and main() both see it

void handler(int signum) {
    running = 0; // just flip it
}

int main(void) {
    /* ... install handler ... */
    while (running) {
        // one unit of work
    }
    // clean up, print results
}
```

Use volatile sig_atomic_t

The only type C guarantees a handler can safely write. volatile makes main() re-read it every loop.

Keep the handler tiny

Just set the flag. printf() isn't async-signal-safe; print from main().

About the starter code

Its plain int keep_going works only because fgetc() forces a re-read. Use the right type anyway.

wc_signal: what happens on Ctrl-C



Typing by hand? `fgetc()` waits until you finish a line, so after Ctrl-C press Enter once more before the counts appear.

wc_signal, after the fix

```
$ make
gcc -Wall -Werror -g -o wc_signal wc_signal.c
$ seq 10 | ./wc_signal
10 lines, 10 words, 21 chars ← input ended
$ yes | ./wc_signal
^C
28725026 lines, 28725027 words, 57450053 chars ← still reported
```

[make test-quiz](#)[make test-code](#)[make test](#)[make zip](#)

Your numbers after Ctrl-C will differ — they depend on when you press it. Answer the quiz in QUESTIONS.txt, then submit the zip to Gradescope.

Any questions?